

PHASE 2 – Recruiter UI + Job Posting Module

1. Phase Objective

Phase 2 extends the secure backend created in Phase 1 and introduces **job posting functionality for recruiters along with a modern UI**. This phase focuses on **both backend and frontend**, ensuring recruiters can create, view, and manage jobs through a real interface.

2. Prerequisites (From Phase 1)

Before starting Phase 2, the following were already completed: - Django project setup - MySQL database connection - Custom User model with roles - JWT authentication - Role-based access control

Phase 2 does **not modify Phase 1 logic**, it only builds on top of it.

3. Technologies & Libraries Used

Backend

- Django
- Django ORM
- MySQL
- Django REST Framework (already installed in Phase 1)

Frontend

- Django Templates
- Bootstrap 5 (via CDN, no installation required)

No new Python libraries were installed in Phase 2.

4. Phase 2 Backend – Job Model

4.1 Why a Job Model is Required

A Job represents a vacancy posted by a recruiter. It stores all information required for: - Displaying jobs - Filtering jobs - Future resume parsing and AI matching

4.2 Job Model Fields (Final)

Field	Purpose
recruiter	Links job to recruiter

Field	Purpose
company_name	Company posting the job
title	Job title
description	Job description
location	Job location
skills_required	Skills for resume matching
experience_required	Experience in years
employment_type	Job type
salary_min	Minimum salary
salary_max	Maximum salary
coding_required	Coding round flag
cutoff_score	Ranking-based cutoff
application_deadline	Last date to apply
status	Active / Closed
created_at	Timestamp

4.3 Job Model Code

File: `accounts/models.py`

```

from django.conf import settings
from django.db import models
from django.utils import timezone

class Job(models.Model):
    STATUS_CHOICES=(
        ('active', 'Active'),
        ('closed', 'Closed'),
    )

    EMPLOYMENT_CHOICES=(
        ('internship', 'Internship'),
        ('full_time', 'Full Time'),
        ('contract', 'Contract'),
    )

    recruiter=models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name='jobs')
    company_name=models.CharField(max_length=255)
    title=models.CharField(max_length=255)

```

```

description=models.TextField()
location=models.CharField(max_length=255)
skills_required=models.TextField()
experience_required=models.IntegerField()

employment_type=models.CharField(max_length=20,choices=EMPLOYMENT_CHOICES)
salary_min=models.IntegerField(null=True,blank=True)
salary_max=models.IntegerField(null=True,blank=True)
coding_required=models.BooleanField(default=False)
cutoff_score=models.IntegerField(null=True,blank=True)
application_deadline=models.DateField()

status=models.CharField(max_length=10,choices=STATUS_CHOICES,default='active')
created_at=models.DateTimeField(default=timezone.now)

def __str__(self):
    return self.title

```

4.4 Database Migration Commands

After adding the model:

```

python manage.py makemigrations
python manage.py migrate

```

This created the `jobs` table in MySQL.

5. Phase 2 Backend – Recruiter Job APIs

5.1 Serializer

File: `accounts/serializers.py`

```

from rest_framework import serializers
from .models import Job

class JobSerializer(serializers.ModelSerializer):
    class Meta:
        model=Job
        fields='__all__'
        read_only_fields=('recruiter','created_at','status')

```

5.2 Views

File: `accounts/views.py`

```
from rest_framework.views import APIView
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework import status
from .models import Job
from .serializers import JobSerializer
from .permissions import IsRecruiter

class RecruiterJobCreateView(APIView):
    permission_classes=[IsAuthenticated,IsRecruiter]

    def post(self,request):
        serializer=JobSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save(recruiter=request.user)
            return Response(serializer.data,status=status.HTTP_201_CREATED)
        return Response(serializer.errors,status=status.HTTP_400_BAD_REQUEST)

class RecruiterJobListView(APIView):
    permission_classes=[IsAuthenticated,IsRecruiter]

    def get(self,request):
        jobs=Job.objects.filter(recruiter=request.user)
        serializer=JobSerializer(jobs,many=True)
        return Response(serializer.data)
```

5.3 API URLs

File: `accounts/urls.py`

```
path('jobs/create/',RecruiterJobCreateView.as_view(),name='job_create'),
path('jobs/my/',RecruiterJobListView.as_view(),name='job_list'),
```

5.4 API Testing (Postman)

Login

```
POST /api/login/
```

Job Creation

Headers:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Body:

```
{
  "company_name": "TechCorp",
  "title": "Backend Developer",
  "description": "Work on APIs",
  "location": "Bangalore",
  "skills_required": "Python,Django,SQL",
  "experience_required": 2,
  "employment_type": "full_time",
  "salary_min": 600000,
  "salary_max": 1200000,
  "coding_required": true,
  "application_deadline": "2026-03-01"
}
```

6. Phase 2 Frontend – UI Setup

6.1 Template Folder Structure

```
accounts/
├── templates/
│   └── accounts/
│       ├── base.html
│       ├── recruiter_dashboard.html
│       └── job_create.html
```

6.2 Base Template

File: `base.html`

```
<!DOCTYPE html>
<html>
<head>
  <title>Recruiter Dashboard</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/
```

```
bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">
<nav class="navbar navbar-dark bg-dark px-4">
    <span class="navbar-brand">Recruiter Portal</span>
    <span class="text-white">{{ request.user.email }}</span>
</nav>
<div class="container py-4">{% block content %}{% endblock %}</div>
</body>
</html>
```

6.3 Recruiter Dashboard View

File: `accounts/views.py`

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render

@login_required
def recruiter_dashboard(request):
    jobs=Job.objects.filter(recruiter=request.user)
    total_jobs=jobs.count()
    active_jobs=jobs.filter(status='active').count()
    closed_jobs=jobs.filter(status='closed').count()

    return render(request, 'accounts/recruiter_dashboard.html', {
        'jobs':jobs,
        'total_jobs':total_jobs,
        'active_jobs':active_jobs,
        'closed_jobs':closed_jobs
    })
```

6.4 Recruiter Dashboard UI

File: `recruiter_dashboard.html`

Includes: - Bootstrap cards for statistics - Job listing table - Status badges - Create Job button

6.5 Job Creation UI

File: `job_create.html`

- Bootstrap form
- Grid layout
- CSRF protection

- Redirect to dashboard on submit
-

6.6 UI URLs

File: `accounts/urls.py`

```
path('dashboard/', recruiter_dashboard, name='recruiter_dashboard'),  
path('jobs/create/ui/', job_create_ui, name='job_create_ui'),
```

7. Phase 2 UI Flow

1. Recruiter logs in (session-based)
 2. Opens dashboard
 3. Views job statistics
 4. Clicks Create Job
 5. Submits job form
 6. Redirected back to dashboard
 7. Job appears in list
-

8. Final Outcome of Phase 2

Phase 2 successfully delivers: - Realistic job model - Secure recruiter-only APIs - Modern Bootstrap-based UI - Recruiter dashboard with statistics - End-to-end job posting through UI

Phase 2 is fully complete and ready for Phase 3 (Candidate Module).